



Tarea 14: Emulación de su computador.

Álvaro Camilo Torres Rodriguez - alvtorresro@unal.edu.co

Sergio Nicolás Correa Escobar - secorreae@unal.edu.co

Juan David Cárdenas Galvis - jcardenasgal@unal.edu.co

Laura Vanesa Alvarez Lafont - lalvarezla@unal.edu.co

Nicolás Andrés Bolaños Fernandez - nbolanosf@unal.edu.co

Santiago Andrés Fetecua Pulgarín - sfetecua@unal.edu.co

Docente

Jorge Eduardo Ortiz Triviño

Universidad Nacional de Colombia

Departamento de Ingeniería de Sistemas e Industrial

Lenguajes de programación

Bogotá D.C., Colombia

2026

1. Marco Teórico.....	3
2. Descripción del Problema.....	4
3. Justificación del Problema.....	4
4. Diseño de Solución.....	5
Registros del Procesador.....	5
5. Código Fuente.....	6
6. Manual de Usuario.....	6
a. Requisitos.....	6
b. Ejecución e Interfaz Principal.....	6
c. Gestión y Edición de Memoria RAM.....	7
7. Manual Técnico.....	8

1. Marco Teórico

La emulación de un computador de propósito general de 64 bits permite materializar mediante software una arquitectura previamente diseñada de forma conceptual. En arquitectura de computadores, un sistema de cómputo se define como una máquina digital capaz de ejecutar programas almacenados en memoria siguiendo un ciclo sistemático de captación, decodificación y ejecución. Este principio se fundamenta en el modelo propuesto por John von Neumann, en el cual datos e instrucciones comparten el mismo espacio de memoria y son procesados secuencialmente por la CPU.

El computador simulado desarrollado adopta una arquitectura de 64 bits, lo que implica que las direcciones, registros y operaciones aritméticas trabajan con palabras de ese tamaño. El sistema se compone de tres subsistemas principales: memoria principal (RAM), procesador (CPU con ALU) y módulo de enlazador–cargador. La memoria tiene una capacidad de 1 MB y se encuentra segmentada en cinco zonas fijas: sistema, código, datos estáticos, heap y pila. Esta organización permite separar instrucciones, datos y memoria dinámica, facilitando la administración y validación de accesos.

El procesador dispone de 16 registros de propósito general y registros especiales como el Contador de Programa (PC), el Puntero de Pila (SP) y el registro de banderas (FLAGS). La Unidad Aritmético-Lógica (ALU) ejecuta operaciones aritméticas, lógicas, comparaciones y desplazamientos. El ciclo de instrucción consiste en obtener la instrucción apuntada por el PC, decodificar su opcode y ejecutar la operación correspondiente, actualizando registros o memoria según sea necesario. Este proceso se repite hasta encontrar la instrucción de parada.

El módulo de enlazador-cargador cumple un papel esencial en la preparación del programa. El enlazador resuelve símbolos y genera un binario absoluto coherente con el espacio de memoria definido. Posteriormente, el cargador copia el código y los datos en las regiones correspondientes e inicializa los registros fundamentales para iniciar la ejecución. Este esquema reproduce, a escala académica, el funcionamiento de sistemas reales que pueden ser emulados mediante herramientas como QEMU.

La implementación del sistema se realiza en Python, lo que permite modelar la memoria, los registros y la ALU mediante estructuras de datos y clases. La organización modular del proyecto, memoria, procesador, enlazador-cargador e ISA, favorece la claridad estructural y la verificación del comportamiento del sistema.

En conclusión, la emulación de este computador de 64 bits integra los fundamentos de la arquitectura de computadores con su implementación práctica. La segmentación de memoria, el ciclo de instrucción y el proceso de enlace y carga permiten transformar el

diseño teórico en un sistema funcional, fortaleciendo la comprensión del funcionamiento interno de un computador real.

2. Descripción del Problema

En la enseñanza de los lenguajes de programación, frecuentemente se estudia la sintaxis, la semántica y la implementación a nivel de software, pero se deja en segundo plano la relación directa entre el código y el hardware que lo ejecuta. Esto genera una brecha conceptual entre cómo se escribe un programa y cómo realmente se procesa en memoria y en el procesador, esto son conocimientos fundamentales para ingenieros de sistemas.

El problema que aborda esta entrega es la necesidad de comprender, desde una perspectiva práctica, el funcionamiento interno de un sistema computacional básico, específicamente la interacción entre la memoria RAM y el procesador, en esta primera entrega. Para ello, se propone el diseño e implementación de los componentes fundamentales de un sistema tipo PC simplificado, comenzando por la construcción de la memoria principal y una unidad central de procesamiento (CPU) con su respectiva ALU, registros y registro de banderas.

Esta primera fase del proyecto se centra en modelar el comportamiento de la RAM como estructura de almacenamiento direccionable y del procesador como unidad capaz de ejecutar instrucciones aritméticas y lógicas mediante códigos de operación definidos (ISA). El reto consiste en lograr que estos componentes interactúen correctamente entre sí, permitiendo la ejecución de instrucciones y la modificación adecuada del estado del sistema (banderas, acumulador y registros).

3. Justificación del Problema

La construcción de un sistema computacional básico, iniciando por la memoria RAM y el procesador, responde a múltiples razones de suma importancia dentro de la formación en lenguajes de programación. Por un lado, permite comprender de manera clara cómo las instrucciones de un ISA se convierten en operaciones binarias ejecutadas por la ALU, afectando registros y banderas de estado. Este enfoque reduce la abstracción excesiva propia de los lenguajes de alto nivel y nos permite ver la lógica interna que sostiene todo algoritmo computacional.

Por otro lado, su implementación resulta pedagógicamente acertada, debido a que diseñar la RAM como estructura direccionable y desarrollar un procesador con acumulador, registro de estado y conjunto básico de instrucciones no solo refuerza conceptos como representación binaria, control de flujo y manejo de desbordamientos, sino que obliga a enfrentar problemas reales de diseño, como definición de códigos de operación, coherencia en la modificación de banderas y correcta interacción entre memoria y CPU, entre otros problemas. De esta manera, el ejercicio deja de ser una simulación teórica y se convierte en una experimentación formativa donde cada decisión impacta el funcionamiento global del sistema computacional que se crea.

Además, abordar este proyecto en etapas progresivas permite construir una comprensión sólida y acumulativa de la arquitectura de un computador, paso a paso. La primera entrega, centrada en la RAM y el procesador, establece la base estructural sobre la cual podrán integrarse posteriormente otras cosas, como memoria secundaria o caché. Así, el trabajo no solo cumple un propósito académico inmediato, sino que fortalece habilidades de diseño, abstracción y pensamiento sistémico, preparándonos para comprender con mayor profundidad tanto la implementación de lenguajes como el desarrollo de sistemas computacionales completos.

4. Diseño de Solución

El diseño y las especificaciones del proyecto **CacaoCore-64** se fundamentan en las directrices establecidas para la tarea 09 de la asignatura Lenguajes de Programación. A continuación, detallamos sus características más relevantes:

Registros del Procesador

Los registros son las únicas variables que el procesador puede usar directamente en sus operaciones. Se decidió por una cantidad de 16 registros, con la necesidad de 4 bits para identificarlos, este número de registros es un equilibrio óptimo, suficiente para que el compilador maneje variables locales cómodamente.

Asignación

#	Nombre	Símbolo	Función y justificación
R0	PC	\$pc	Program Counter. Dirección de la próxima instrucción a ejecutar. El simulador lo incrementa automáticamente y lo modifica en saltos y llamadas a función.
R1	SP	\$sp	Stack Pointer. Apunta a la cima de la pila en RAM. Se decrementa al guardar datos (PUSH) y se incrementa al recuperarlos (POP). Esencial para llamadas a funciones.
R2	LR	\$lr	Link Register. Guarda la dirección de retorno cuando se llama a una función (CALL). Al terminar la función, el PC se restaura desde LR (RET).
R3	FLAGS	\$fl	Registro de banderas. Sus bits individuales registran el resultado de la última operación: Z (resultado cero), N (negativo), V (desbordamiento), C (acarreo). Los saltos condicionales leen este registro.
R4–R15	GP0–GP11	\$r4–\$r15	Registros de propósito general. El compilador y el programador los usan libremente para variables locales, parámetros de funciones, resultados intermedios y contadores de bucles. Son 12 registros libres en total.

Mapa de Memoria RAM: División del Espacio de 1 MB

Zona	Rango de direcciones	Tamaño	Justificación
Sistema / Vectores	0x00000000 – 0x00000FFF	4 KB	Reservado para el simulador. El PC se inicializa en 0x00001000 (inicio del código). Protegido contra escritura del programa de usuario.
Código del programa	0x00001000 – 0x0003FFFF	252 KB	Las instrucciones compiladas se cargan aquí. Zona de solo lectura durante la ejecución. 252 KB permite programas de tamaño considerable.
Datos estáticos	0x00040000 – 0x0007FFFF	256 KB	Variables globales y constantes. Persisten durante toda la ejecución del programa.
Heap (datos dinámicos)	0x00080000 – 0x000BFFFF	256 KB	Memoria dinámica (equivalente a malloc/new). Crece desde 0x00080000 hacia direcciones más altas.

Pila (Stack)	0x000C0000 – 0x000FFFFF	256 KB	Variables locales, parámetros y direcciones de retorno. SP se inicializa en 0x000FFFFF y crece hacia abajo. 256 KB da margen amplio incluso para algoritmos con alta profundidad de recursión.
--------------	-------------------------	--------	--

Formato de instrucciones de CACAO_Core-64

Las instrucciones de CACAO_Core-64 se constituyen por un código de operación y, a lo más, dos operandos. Las instrucciones pueden seguir cualquiera de las siguientes estructuras:

- Operación
- Operación Operando
- Operación Operando_destino, Operando_fuente

Modos de direccionamiento:

- **Inmediato**: En este modo de direccionamiento, el operando se incluye directamente como un valor constante, que no requiere de un registro ni dirección de memoria.
- **Directo**: En este modo de direccionamiento, el operando es la dirección de memoria la cual se quiere acceder esta debe estar en el rango válido definido para la memoria. Posteriormente se va a esa dirección y se recupera el contenido de esta.
- **Indirecto**: En este modo de direccionamiento, el operando hace referencia a uno de los registros de propósito general en el que se almacenó previamente una dirección de memoria, dirección en la que se almacena el valor a utilizar en la operación.

5. Código Fuente

El código fuente se encuentra adjunto en el entregable 14 Fetecua Pulgarin Santiago Andres.zip

6. Manual de Usuario

CacaoCore-64 es un emulador educativo de un procesador de 64 bits que implementa una arquitectura Von Neumann con un espacio de memoria RAM de 1 MB y un conjunto de instrucciones personalizado. La herramienta permite comprender el funcionamiento interno de un procesador mediante la visualización y manipulación directa de registros, memoria y flujo de ejecución. El presente manual describe los requisitos, el procedimiento de ejecución, el uso de la interfaz gráfica y las pautas básicas de operación y resolución de problemas.

a. Requisitos

Para ejecutar el emulador se requiere:

- Python 3.8 o superior.
- Tkinter (incluido por defecto en la mayoría de instalaciones de Python).
- Aproximadamente 10 MB de espacio en disco.
- Resolución recomendada: 1920x1080.

Para verificar Python, se puede usar el comando “python --version”.

b. Ejecución e Interfaz Principal

Desde la carpeta raíz del proyecto, después de descomprimir el archivo .zip: “python -m src.cacao_gui”.

La interfaz se abrirá automáticamente en pantalla, para salir, cierre la ventana.

La interfaz principal constituye el núcleo de interacción con el procesador simulado. En ella se visualizan los registros fundamentales del sistema, los controles de ejecución y las opciones de visualización de datos.

En la sección superior se muestran los registros de control más relevantes: el Program Counter (PC), que indica la dirección de la siguiente instrucción a ejecutar; el Instruction Register (IR), que contiene la instrucción actualmente en proceso; y el Flags Register (FR), que almacena indicadores de estado generados por operaciones aritméticas y lógicas. Estos indicadores incluyen las banderas de interrupción (I), desbordamiento (V), acarreo (C), negativo (N) y cero (Z), las cuales permiten evaluar condiciones posteriores a una operación.

Debajo de estos registros se encuentran los 16 registros de propósito general de 64 bits. Los registros r0 a r12 pueden emplearse libremente para almacenamiento temporal de datos. El registro r13 corresponde al Stack Pointer (\$sp), que apunta al tope de la

pila; r14 es el Link Register (\$lr), utilizado para almacenar direcciones de retorno en llamadas a funciones; y r15 es el Acumulador (\$acc), empleado por la unidad aritmético-lógica para almacenar resultados.

La interfaz incorpora tres controles principales de ejecución. El botón BOOT reinicia completamente el procesador, limpia los registros y posiciona el Program Counter en la dirección inicial especificada por el usuario. El botón RUN FULL ejecuta el programa de manera continua hasta encontrar una instrucción HLT o producirse un error. El botón RUN STEP permite ejecutar una única instrucción por vez, facilitando la depuración detallada del código.

El usuario debe especificar la dirección de inicio en formato hexadecimal, sin prefijo "0x". Asimismo, puede seleccionar el formato de visualización de los registros entre hexadecimal, decimal o binario. Esta selección únicamente afecta la representación visual de los datos y no altera su valor interno.

c. Gestión y Edición de Memoria RAM

El emulador dispone de un editor de memoria RAM accesible desde la interfaz principal. La memoria total es de 1 MB y se encuentra segmentada lógicamente en zonas destinadas a sistema, código, datos estáticos, heap y stack.

La zona de código comienza en la dirección hexadecimal 0x00001000, siendo el punto habitual para cargar programas. El usuario puede introducir los bytes manualmente, cargarlos desde un archivo en formato binario o hexadecimal, o modificarlos directamente por dirección. Cada celda representa un byte expresado en dos dígitos hexadecimales.

Una vez cargado el programa, la información permanece en memoria incluso después de cerrar el editor, permitiendo regresar a la interfaz principal para iniciar la ejecución.

d. Carga y Ejecución

Para ejecutar un programa, el usuario debe cargar previamente el código en la zona correspondiente de la memoria. Posteriormente, debe especificar la dirección inicial de ejecución (por ejemplo, 1000 en hexadecimal) en el campo "Start Address" y presionar BOOT para reiniciar el sistema.

Una vez inicializado el procesador, puede optarse por ejecutar el programa completo mediante RUN FULL o proceder paso a paso con RUN STEP. Durante la ejecución, se recomienda observar la evolución del Program Counter, los cambios en los registros generales y la activación de las banderas del registro de estado, especialmente en operaciones aritméticas o comparaciones.

En caso de requerir verificación adicional, el Editor de RAM puede abrirse nuevamente para inspeccionar los datos almacenados o modificados durante la ejecución.

e. Consideraciones

Si el programa no avanza o parece detenido, es probable que se encuentre en un bucle infinito o que no contenga una instrucción HLT. En estos casos se recomienda reiniciar con BOOT y utilizar el modo RUN STEP para identificar el punto de bloqueo. Si el Program Counter no cambia tras ejecutar una instrucción, debe verificarse que el sistema haya sido reiniciado correctamente y que la dirección de inicio sea válida y esté en formato hexadecimal.

Cuando los registros presentan valores inesperados, la estrategia adecuada consiste en ejecutar el programa paso a paso para determinar qué instrucción genera la modificación no deseada. También es recomendable revisar las direcciones de memoria utilizadas. Los errores de tipo “Address Out of Range” indican que el programa intenta acceder a direcciones fuera del rango válido de la RAM (0x00000000 a 0x000FFFFF). Deben revisarse los operandos y cálculos de dirección.

Si el campo de dirección inicial no acepta la entrada, debe comprobarse que únicamente se utilicen caracteres hexadecimales válidos (0–9 y A–F) sin prefijo adicional. Finalmente, si la interfaz gráfica presenta problemas de visualización, se recomienda ajustar la resolución del sistema o revisar la configuración de escalado del sistema operativo.

CacaoCore-64 está diseñado como herramienta académica para el estudio de arquitectura de computadores. El uso del modo paso a paso, la observación sistemática de registros y flags, y la verificación periódica de la memoria constituyen buenas prácticas para la comprensión profunda del comportamiento del procesador.

Este entorno permite experimentar de manera controlada con conceptos fundamentales como ejecución secuencial, manejo de pila, direccionamiento de memoria y control de flujo.

7. Manual Técnico

El presente documento describe la arquitectura interna, organización modular y principales interfaces de programación del emulador CacaoCore-64. Está dirigido a desarrolladores que requieran comprender el diseño del sistema, extender su funcionalidad o realizar procesos de depuración a nivel de código fuente.

a. Organización General

El proyecto se encuentra estructurado en torno a una organización modular dentro del directorio src, separando claramente la lógica del procesador, la gestión de memoria, el conjunto de instrucciones (ISA) y la interfaz gráfica. Esta separación favorece el mantenimiento, la extensibilidad y la comprensión conceptual del sistema.

El punto de entrada de la aplicación es cacao_core.py, que inicializa la interfaz gráfica y crea la instancia principal de la unidad de control. El paquete processor contiene el núcleo de simulación del procesador (Control Unit, ALU, decodificador y microinstrucciones). El paquete memoria implementa la RAM y su editor gráfico. El paquete isa documenta y define el formato de las microinstrucciones.

El sistema está implementado completamente en Python 3 y utiliza de manera intensiva la estructura bytearray para representar registros y memoria, permitiendo manipulación directa de datos binarios de longitud fija.

b. Núcleo del Procesador

El componente central del emulador es la clase ControlUnit, definida en control_unit.py. Esta clase modela el ciclo clásico fetch-decode-execute y mantiene el estado completo del procesador, incluyendo registros generales, registros de control, flags y estado de ejecución (RUNNING o HALTED).

Internamente, los registros generales (r0–r15) se almacenan como una lista de bytearray de 8 bytes cada uno. El Program Counter, Instruction Register, Memory Address Register, Memory Data Register y Flags Register también se representan mediante bytearray, respetando su tamaño lógico. Los flags se gestionan bajo convención little-endian, con un mapeo explícito de bits para Z, N/S, C, V e I.

El método boot(start_address: int) reinicia completamente el sistema, limpia registros, inicializa el stack pointer y posiciona el PC en la dirección indicada. Los métodos run_full_exec() y run_step() controlan la ejecución continua o paso a paso respectivamente. Las fases internas _fetch(), _decode() y _execute() encapsulan el ciclo de instrucción, gestionando MAR, MDR, IR y la invocación dinámica de microinstrucciones.

La clase ALU, definida en `alu.py`, implementa las operaciones aritmético-lógicas y actualiza automáticamente el registro de flags en función del resultado. Opera directamente sobre bytearray y mantiene una referencia al acumulador y al registro de estado. Las operaciones como suma, resta, multiplicación y operaciones lógicas modifican los bits Z, N, C y V conforme a la semántica del ISA.

El proceso de decodificación está a cargo de la clase Decoder, que interpreta el contenido del Instruction Register utilizando un puntero interno de decodificación. El resultado es un identificador de microinstrucción que la Control Unit utiliza para ejecutar dinámicamente el método correspondiente, apoyándose en el mapa generado por `instruction_map.py`.

Las implementaciones concretas de las microinstrucciones residen en `microinstructions_mixin.py`, que se integra mediante herencia múltiple en la ControlUnit. Este enfoque permite mantener la lógica del ciclo separada de la semántica específica de cada instrucción, favoreciendo la extensibilidad.

c. Sistema de Memoria

La memoria RAM se modela como un bloque lineal de 1 MB administrado por la clase RAM, definida en `ram.py`. Esta clase encapsula el acceso seguro mediante métodos `write()` y `read()`, que validan límites de dirección, alineación y permisos de escritura.

Se definen excepciones específicas como `AddressOutOfRange`, `AlignmentError` y `WriteProtectionError`, permitiendo un manejo explícito de errores durante el acceso a memoria. Las distintas zonas (sistema, código, datos, heap y stack) se definen mediante constantes de inicio y fin, y son utilizadas tanto por la Control Unit como por la interfaz gráfica.

La instancia de RAM se exporta globalmente como `ram`, lo cual simplifica el acceso desde los diferentes módulos del sistema.

El editor gráfico de memoria (`cacao_ram_editor.py`) proporciona una interfaz basada en Tkinter para visualizar y modificar la RAM. Implementa tablas de bytes, navegación por dirección, carga desde archivo y validación de entrada. Internamente encapsula las llamadas a `ram.read()` y `ram.write()`, gestionando excepciones para evitar que errores internos afecten la estabilidad de la interfaz.

d. Conjunto de Instrucciones

El paquete `isa` contiene la definición formal del conjunto de microinstrucciones soportado por el emulador. El archivo `microinstructions.py` define los formatos binarios y estructuras necesarias para la decodificación, mientras que el README asociado actúa como referencia oficial del ISA, incluyendo descripciones de operandos y ejemplos de codificación.

El decodificador depende de esta información para interpretar correctamente los campos de cada instrucción y dirigir la ejecución hacia el método correspondiente.

e. Integración

El módulo `cacao_core.py` inicializa la aplicación Tkinter y construye la ventana principal en modo pantalla completa. En este archivo se instancia la `ControlUnit` y se establecen los vínculos entre los botones de la interfaz y los métodos de ejecución (`boot`, `run_full_exec`, `run_step`).

La función `refresh_registers()` consulta periódicamente `cu.get_registers()` y actualiza los widgets visuales según el formato seleccionado por el usuario. Esta actualización se ejecuta tanto tras cada ciclo de ejecución como mediante llamadas programadas con `after()`, garantizando reactividad sin bloquear el hilo principal.

El sistema utiliza `ttk.Style()` para aplicar una paleta oscura coherente con el editor de memoria. La validación del campo de dirección inicial se realiza mediante expresiones regulares y conversión explícita de hexadecimal a entero antes de invocar `boot()`.

Dado que Tkinter no es thread-safe, toda interacción con la interfaz debe realizarse desde el hilo principal. Para futuras extensiones con ejecución en segundo plano, se recomienda emplear colas y sincronización mediante `after()`.

Desde el punto de vista programático, los principales puntos de extensión son la clase `ControlUnit` y la instancia global `ram`. La Control Unit expone métodos como `boot()`, `run_full_exec()`, `run_step()` y `get_registers()`, además de utilidades de conversión como `bytes_to_int()`.

Para extender el conjunto de instrucciones, el desarrollador debe definir el formato correspondiente en isa, implementar la semántica en `microinstructions_mixin.py` y asegurar que el método sea detectado por el mecanismo de mapeo dinámico.

Las modificaciones de interfaz deben realizarse en `cacao_core.py` o en los módulos auxiliares de GUI, evitando introducir lógica pesada en el hilo principal.

f. Consideraciones

El emulador prioriza claridad estructural sobre rendimiento. El uso intensivo de conversiones entre `bytearray` e enteros implica un costo computacional significativo en ejecuciones prolongadas. En escenarios de optimización futura podrían evaluarse alternativas como el uso de `struct` o buffers prealineados.

Para depuración, puede utilizarse instrumentación mediante `print()` o el módulo `logging` dentro de la Control Unit. Asimismo, es posible inspeccionar directamente el contenido de memoria accediendo a `ram._mem` desde el intérprete Python.

Se recomienda la incorporación de pruebas automatizadas mediante pytest para validar el comportamiento de la ALU, la correcta activación de flags y la coherencia del ciclo fetch-decode-execute.

El proyecto se distribuye bajo la licencia especificada en el archivo LICENSE. Cualquier redistribución o modificación debe respetar sus términos. Finalmente, el Manual Técnico complementa el Manual de Usuario proporcionando una visión estructural y de implementación del sistema. Su comprensión resulta fundamental para extender el emulador, incorporar nuevas microinstrucciones o adaptar la arquitectura a fines académicos o experimentales.

8. Experimentación y Análisis de Resultados

Para probar la máquina implementada se ejecutaron los siguientes algoritmos escritos en código máquina donde se vio el paso a paso de su ejecución:

Bubble Sort

None

```
05 00 00 00 04 00 19 F0
03 00 08 00 04 00 19 F0
08 00 10 00 04 00 19 F0
01 00 18 00 04 00 19 F0
04 00 20 00 04 00 19 F0
00 00 04 00 20 15 FF FF
05 00 11 01 FF FF FF FF
00 00 12 01 FF FF FF FF
81 10 F0 FF FF FF FF FF
48 52 FF FF FF FF FF FF
28 B0 60 FF FF FF FF FF
F8 10 00 00 8E FF FF FF
00 00 13 01 FF FF FF FF
82 50 10 FF FF FF FF FF
38 B0 60 FF FF FF FF FF
E8 10 00 00 8E FF FF FF
63 10 F0 FF FF FF FF FF
08 00 16 0B F3 FF FF FF
60 40 00 FF FF FF FF FF
76 10 F0 FF FF FF FF FF
08 00 17 09 F0 FF FF FF
46 03 F0 FF FF FF FF FF
57 03 F0 FF FF FF FF FF
45 B0 60 FF FF FF FF FF
D8 10 00 00 96 FF FF FF
65 04 FF FF FF FF FF FF
74 04 FF FF FF FF FF FF
```

```
03 22 FF FF FF FF FF FF
70 10 00 00 52 FF FF FF
02 22 FF FF FF FF FF FF
50 10 00 00 52 FF FF FF
00 00 13 01 FF FF FF FF
01 00 11 0A F1 FF FF FF
31 B0 60 FF FF FF FF FF
68 11 00 00 8E FF FF FF
43 10 F0 FF FF FF FF FF
08 00 14 0B F3 FF FF FF
40 40 00 FF FF FF FF FF
24 03 F0 FF FF FF FF FF
02 D0 00 00 80 02 FF FF
00 0D 00 00 2B 15 FF FF
08 00 1C 02 FF FF FF FF
00 00 45 FF FF FF FF FF
03 22 FF FF FF FF FF FF
08 11 00 00 52 FF FF FF
00 00 00 00 00 00 00 00
```

Búsqueda mayor

None

```
01 00 00 00 04 00 19 F0
08 00 08 00 04 00 19 F0
04 00 10 00 04 00 19 F0
02 00 18 00 04 00 19 F0
0A 00 20 00 04 00 19 F0
05 00 28 00 04 00 19 F0
28 00 04 00 20 01 FF FF
00 00 11 01 FF FF FF FF
00 00 04 00 22 01 FF FF
```

```
01 22 FF FF FF FF FF FF
40 52 FF FF FF FF FF FF
10 B0 60 FF FF FF FF FF
B8 10 00 00 8E FF FF FF
41 30 F0 FF FF FF FF FF
08 00 14 0B F3 FF FF FF
00 00 04 00 25 15 FF FF
45 40 00 FF FF FF FF FF
64 13 F0 FF FF FF FF FF
62 B0 60 FF FF FF FF FF
A8 10 00 00 96 FF FF FF
26 30 F0 FF FF FF FF FF
01 22 FF FF FF FF FF FF
58 10 00 00 52 FF FF FF
02 D0 00 00 80 02 FF FF
00 0D 00 00 2B 15 FF FF
08 00 1C 02 FF FF FF FF
00 00 45 FF FF FF FF FF
00 00 00 00 00 00 00 00
```

CACAO_Core-64 - Panel de Control

CACAO_Core-64 PANEL DE CONTROL 64-bit · Von Neumann · 1 MB RAM | cacao_core.py OK

CONTROL DE EJECUCION

Start Address: 0x 00001000

BOOT

RUN FULL

RUN STEP

PROGRAM COUNTER [PC]

0x0000000000001070

INSTRUCTION REG [IR]

0x0000000000000000

REGISTROS ESPECIALES

\$sp Stack Pointer 0x000000000000FFFF8

\$lr Link Register 0x0000000000000000

\$acc Acumulador 0x0000000000000010

REGISTROS INTERNOS (MAR · MDR · DP)

MAR MDR DP

0x00001068 0x000000000000 0x01

FLAGS REGISTER [FR] - little-endian

FR raw: 0x10 · 0b00010000

Z Zero 1 bit 4	N Negative 0 bit 3	C Carry 0 bit 2	V Overflow 0 bit 1	I Interrupt 0 bit 0
--------------------------------	------------------------------------	---------------------------------	------------------------------------	-------------------------------------

MEMORIA RAM

Accede al editor directo de RAM para inspeccionar, cargar y modificar el contenido de memoria. Se abre como ventana independiente.

ABRIR RAM EDITOR

REGISTROS GPR (r0 - r12)

\$r0	0x0000000000000006
\$r1	0x0000000000000004
\$r2	0x0000000000000018
\$r3	0x0000000000000000
\$r4	0x0000000000000000
\$r5	0x0000000000000000
\$r6	0x0000000000000000
\$r7	0x0000000000000000
\$r8	0x0000000000000000
\$r9	0x0000000000000000
\$r10	0x0000000000000000
\$r11	0x0000000000000000

CONSOLA DE SISTEMA 10 líneas

☒ Auto-scroll Filtro: **INFO** **OK** **WARN** **ERR** **CLR**

```

15:47:56.172 [i] CACAO_Core-64 · Consola de Sistema
15:47:56.172 [i] Sistema listo. Esperando instrucciones.

15:48:07.553 [✓] BOOT completado · PC -> 0x00001000
15:48:07.559 [H] PC: 0x00001000
15:48:09.071 [✓] RAM Editor abierto en proceso compart
15:48:42.384 [✓] RAM Editor abierto en proceso compart
15:48:46.583 [✓] 24
15:48:46.587 [✓] RUN FULL completado · procesador de
  
```

Multiplicación

None

```

06 00 10 01 FF FF FF FF
04 00 11 01 FF FF FF FF
00 00 12 01 FF FF FF FF
31 10 F0 FF FF FF FF FF
00 00 13 10 F6 FF FF FF
48 10 00 00 96 FF FF FF
20 40 00 FF FF FF FF FF
43 52 FF FF FF FF FF FF
20 10 00 00 52 FF FF FF
02 D0 00 00 80 02 FF FF
00 0D 00 00 2B 15 FF FF
08 00 1C 02 FF FF FF FF
00 00 45 FF FF FF FF FF
00 00 00 00 00 00 00
  
```

The screenshot displays the CACAO_Core-64 control panel interface. The top bar shows the title 'CACAO_Core-64 - Panel de Control' and the status '64-bit · Von Neumann · 1 MB RAM · cacao_core.py OK'.

CONTROL DE EJECUCION: The 'Start Address' is set to 0x00001000. The 'BOOT' button is highlighted. Below it are 'RUN FULL' and 'RUN STEP' buttons.

FORMATO DE REGISTROS: The 'Base numerica para todos los registros:' is set to DEC. Buttons for DEC, BIN, and OCT are visible.

REGISTROS GPR (r0 - r12): A table showing the values of General Purpose Registers. The values are mostly 0x0000000000000000, with r0 being 0x0000000000000004.

PROGRAM COUNTER [PC]: The value is 0x00000000000010E0.

INSTRUCTION REG [IR]: The value is 0x0000000000000000.

REGISTROS ESPECIALES: A table showing special registers: \$sp (Stack Pointer) is 0x000000000000FFFF8, \$lr (Link Register) is 0x0000000000000000, and \$acc (Acumulador) is 0x0000000000000010.

REGISTROS INTERNOS (MAR · MDR · DP): A table showing internal registers: MAR is 0x000010D8, MDR is 0x0000000000000000, and DP is 0x01.

FLAGS REGISTER [FR] - little-endian: A table showing flag bits: Z (Zero) is 0, N (Negative) is 0, C (Carry) is 0, V (Overflow) is 0, and I (Interrupt) is 0.

MEMORIA RAM: A section with a description: 'Accede al editor directo de RAM para inspeccionar, cargar y modificar el contenido de memoria. Se abre como ventana independiente.' and a button 'ABRIR RAM EDITOR'.

CONSOLA DE SISTEMA: A log window showing system messages. The messages include: 'CACAO_Core-64 · Consola de Sistema', 'Sistema listo. Esperando instrucciones.', 'BOOT completado · PC -> 0x00001000', 'PC: 0x00001000', 'RAM Editor abierto en proceso de compar', '10', and 'RUN FULL completado · procesador de'.

Tras estas ejecuciones, en consola se observan los resultados de los algoritmos; se comprobó que el computador se comporta según lo esperado y realiza las instrucciones usadas en los programas correctamente.